

Paterni ponašanja

1. Strategy patern

Strategy patern koristi se kada postoji više različitih algoritama za rješavanje istog problema, pri čemu se konkretni algoritam može birati u zavisnosti od potrebe korisnika ili trenutnog konteksta. Umjesto da se svi algoritmi nalaze u jednoj klasi, oni se izdvajaju u posebne klase, čime se kod čini preglednijim i lakšim za proširenje.

U aplikaciji Optika Oculis ovaj patern je pogodan za implementaciju različitih algoritama obračuna popusta. Sistem može nuditi popuste na osnovu tipa korisnika, količine narudžbe ili sezonskih akcija. Klasa ObracunPopusta može koristiti različite strategije kao što su FixniPopust, ProcentualniPopust ili LojalnosniPopust. Svaki od ovih načina obračuna može se izdvojiti kao posebna strategija, čime se novi tip popusta može dodati bez izmjene glavne logike narudžbe i bez narušavanja Open/Closed principa.

2. State patern

State patern omogućava da se ponašanje objekta mijenja u zavisnosti od njegovog trenutnog stanja. Umjesto da se u kodu stalno provjerava stanje pomoću velikog broja if ili switch naredbi, svako stanje može imati posebno definisano ponašanje.

U aplikaciji Optika Oculis ovaj patern se prirodno primjenjuje na klasu Narudzba. Narudžba može prolaziti kroz stanja definisana enumeracijom StatusNarudzbe: Zaprimljena, UObradi, Poslana, Isporucena i Otkazana. Od trenutnog stanja zavisi koje su akcije dozvoljene — zaprimljena narudžba može biti obrađena, dok otkazana narudžba ne može biti isporučena. Korištenjem State paternu svako stanje narudžbe može se jasno izdvojiti i zasebno obrađivati, čime se eliminišu složena uslovana grananja unutar klase Narudzba.

3. Template Method patern

Template Method patern definiše osnovnu strukturu određenog procesa, dok se pojedini koraci tog procesa mogu mijenjajući implementirati u izvedenim klasama. Time se izbjegava ponavljanje zajedničke logike, a istovremeno se ostavlja mogućnost prilagođavanja specifičnih dijelova algoritma.

U aplikaciji Optika Oculis ovaj patern se može primijeniti kod procesa plaćanja. Klasa Placanje definiše zajednički tok: validacija iznosa, provjera dostupnosti, izvršavanje plaćanja i ažuriranje statusa narudžbe. Konkretno Klase KarticnoPlacanje i GotovinskoPlacanje implementiraju specifične detalje svakog načina plaćanja kroz metod izvrsiPlacanje. Zajednički tok obrađuje se na jednom mjestu, dok se specifična logika za svaki tip plaćanja odvija u podklasama, čime se izbjegava dupliranje koda.

4. Observer patern

Observer patern uspostavlja vezu između objekta čije se stanje prati i objekata koji trebaju biti obaviješteni kada dođe do promjene tog stanja. Ovaj patern je posebno koristan kada jedna promjena u sistemu treba izazvati više različitih reakcija.

U aplikaciji Optika Oculis najprirodnija primjena Observer paternu odnosi se na sistem obavijesti vezan uz klasu ObavijestService. Kada se promijeni status narudžbe, sistem

automatski obavještava korisnika putem klase Obavijest. Klasa Narudzba ima ulogu Subject objekta, dok klasa Korisnik predstavlja Observer objekt koji prima notifikaciju. Time se logika upravljanja narudžbom ne mora direktno vezati za način slanja obavijesti, čime se postiže labavo povezivanje između modula Narudzba i Obavijest.

5. Iterator patern

Iterator patern omogućava prolazak kroz kolekciju objekata bez potrebe da se pozna njena unutrašnja struktura. Klijent koristi jedinstven način pristupa elementima, bez obzira na to da li se podaci nalaze u listi, nizu, skupu ili nekoj drugoj strukturi.

U aplikaciji Optika Oculis ovaj patern može biti koristan kod prikaza različitih kolekcija podataka. Sistem sadrži kolekcije kao što su lista proizvoda, lista stavki korpe, lista narudžbi korisnika, lista termina pregleda i lista obavijesti. Iterator bi omogućio ujednačen način prolaska kroz te podatke, posebno kod prikaza historije narudžbi, generisanja izvještaja ili prikaza dostupnih termina u klasi TerminPregleda, bez obzira na internu strukturu kolekcija.

6. Command patern

Command patern pretvara zahtjev u poseban objekat. Na taj način se akcije mogu evidentirati, odgoditi, ponoviti ili poništiti. Ovaj patern je koristan u sistemima gdje se korisničke akcije trebaju jasno pratiti ili gdje postoji potreba za logovanjem aktivnosti.

U aplikaciji Optika Oculis se kao komande mogu posmatrati akcije poput dodavanja proizvoda u korpu, kreiranja narudžbe, zakazivanja termina pregleda i izvršavanja plaćanja. Svaka od tih operacija ima svoje ulazne podatke i posljedice po sistem. Ako bi se one izdvojile u posebne command klase poput DodajUKorpuCommand, KreirajNarudžbuCommand i ZakaziTerminCommand, bilo bi lakše evidentirati akcije korisnika i implementirati funkcionalnost poništavanja pogrešno izvršene operacije.

7. Chain of Responsibility patern

Chain of Responsibility patern omogućava da se zahtjev prosljeđuje kroz niz objekata, pri čemu svaki objekat provjerava da li može obraditi zahtjev ili ga prosljeđuje dalje. Ovaj patern je koristan kada se nad jednim zahtjevom mora izvršiti više provjera.

U aplikaciji Optika Oculis ovaj patern se može primijeniti kod obrade zahtjeva za kreiranje narudžbe. Prije nego što sistem potvrdi narudžbu, potrebno je provjeriti više uslova: da li korisnik ima aktivan račun, da li su svi traženi proizvodi dostupni na stanju (klasa Proizvod, atribut dostupnaKolicina), da li je korpa neprazna i da li je plaćanje uspješno izvršeno. Svaka od ovih provjera može biti jedan član lanca, čime se nova provjera može dodati bez izmjene postojećeg toka obrade.

8. Mediator patern

Mediator patern uvodi centralni objekat koji upravlja komunikacijom između više drugih objekata. Umjesto da objekti direktno zavise jedni od drugih, oni komuniciraju preko posrednika. Time se smanjuje međusobna povezanost klasa i olakšava održavanje sistema.

U aplikaciji Optika Oculis klasa NarudzbaFacade preuzima ulogu mediatora koji koordinira komunikaciju između modula narudžbe, plaćanja i obavijesti. Kada korisnik potvrdi narudžbu, NarudzbaFacade poziva metod obradiNarudzbu koji koordinira:

ažuriranje statusa narudžbe, iniciranje plaćanja kroz klasu Placanje, obračun popusta kroz klasu ObracunPopusta i slanje obavijesti korisniku kroz ObavijestService. Time klase ne moraju direktno komunicirati jedna s drugom, što smanjuje zavisnosti i poboljšava modularnost sistema.

9. Visitor patern

Visitor patern omogućava dodavanje novih operacija nad postojećim objektima bez izmjene njihovih klasa. Koristan je kada sistem ima stabilnu strukturu objekata, ali se nad njima često žele izvršavati različite dodatne operacije.

U aplikaciji Optika Oculis ovaj patern se može koristiti kod generisanja izvještaja i statističkih analiza. Isti skup objekata kao što su Proizvod, Narudzba, Korisnik i ElektronskiKarton može se obrađivati na različite načine. Administrator može željeti izvještaj o najprodavanijim proizvodima po kategorijama (SuncaneNaocale, DioptrijskeNaocale), menadžer izvještaj o ukupnoj vrijednosti narudžbi, a ljekar pregled historije vidnih pregleda. Visitor bi omogućio da se ove operacije dodaju bez direktnog mijenjanja osnovnih modela.

10. Memento patern

Memento patern omogućava čuvanje prethodnog stanja objekta kako bi se ono kasnije moglo vratiti. Ovaj patern je koristan u situacijama gdje postoji mogućnost greške pri izmjeni podataka ili gdje korisnik treba imati mogućnost vraćanja prethodne verzije.

U aplikaciji Optika Oculis ovaj patern se može primijeniti kod izmjene podataka u elektronskom kartonu pacijenta. Klasa ElektronskiKarton sadrži osjetljive medicinske podatke, a uz nju je vezana klasa PregledVida koja čuva vrijednosti dioptrije i preporuke. Prije nego što ljekar izmijeni dijagnozu ili vrijednosti dioptrije, sistem može sačuvati prethodno stanje objekta PregledVida. Ukoliko se utvrdi da su uneseni pogrešni podaci, moguće je vratiti prethodnu verziju pregleda bez ručnog ponovnog unosa, što povećava sigurnost rada s medicinskim podacima.